

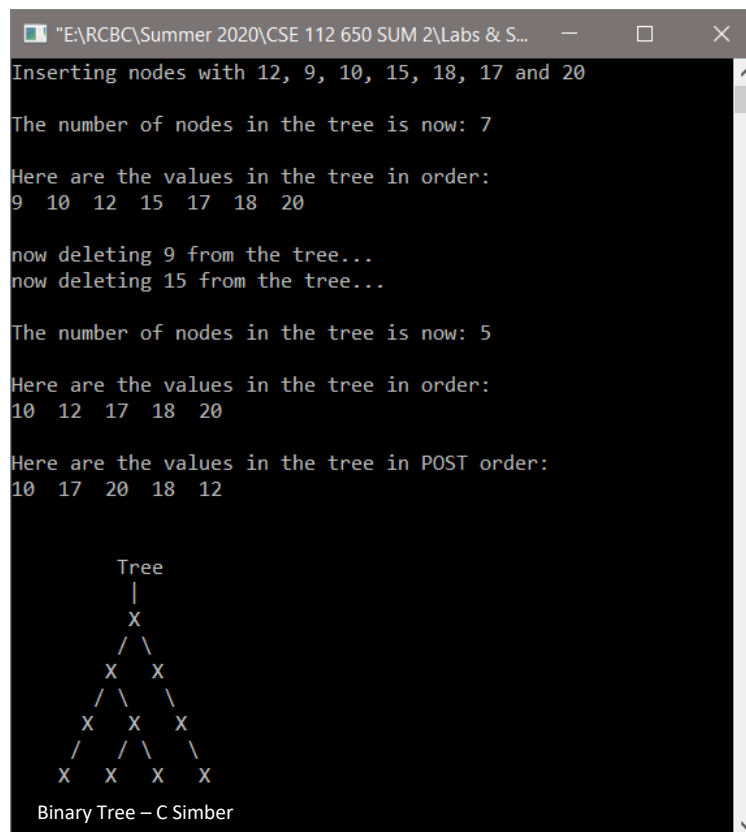
CSE 112 – Lab #12 – Templates, Binary Search Trees and Operations

Using the BinaryTree.h file (shown below and posted as a text file on BB), write a program that inserts the following numbers 12, 9, 10, 15, 18, 17 and 20 into the tree in that order.

- Announce the numbers being inserted
- Output the number of nodes, and then display them in order
- Announce the deletion of the numbers 9 and 15 from the tree
- Output the number of nodes, and then display them in order
- Display the tree in postorder, and then “Binary Tree - *your name*”
- Modify the header as needed to output the values horizontally
- **Extra Credit (10)** - Display the tree as it exists at the end replacing the “X”s shown below with the correct node values. Use “cout” statements and only display branches and numbers that exist, then “Binary Tree – first initial last name” as shown below.

Submit the main program as a text file.

NOTE: Older compilers do not accept nullptr, use NULL to initialize pointers. Ref: Page 500



```
"E:\RCBC\Summer 2020\CSE 112 650 SUM 2\Labs & S...  —  □  ×
Inserting nodes with 12, 9, 10, 15, 18, 17 and 20

The number of nodes in the tree is now: 7

Here are the values in the tree in order:
9 10 12 15 17 18 20

now deleting 9 from the tree...
now deleting 15 from the tree...

The number of nodes in the tree is now: 5

Here are the values in the tree in order:
10 12 17 18 20

Here are the values in the tree in POST order:
10 17 20 18 12

      Tree
      |
      X
     / \
    X   X
   / \ / \
  X  X X  X
 / \ / \ / \
X  X X  X X  X

Binary Tree – C Simber
```

Grading will include meeting all of the lab requirements, adherence to programming standards, operation and accurate output, and programming style including white space and indentation.

Assignments submitted after the due date will incur a grade penalty of three (3) points per day after the first day, and will not be accepted more than one (1) week late.

```
// Binary Tree Template
#ifndef BINARYTREE_H
#define BINARYTREE_H
#include <iostream>
using namespace std;

template <class T>
class BinaryTree
{
public:
    struct TreeNode
    {
        T value;
        TreeNode *left;
        TreeNode *right;
    };

    TreeNode *root;
    void insert(TreeNode *&, TreeNode *&);
    void destroySubTree(TreeNode *);
    void deleteNode(T, TreeNode *&);
    void makeDeletion(TreeNode *&);
    void displayInOrder(TreeNode *);
    void displayPreOrder(TreeNode *);
    void displayPostOrder(TreeNode *);
    int countNodes(TreeNode *&);

public:
    BinaryTree()    // Constructor
        { root = nullptr; }
    ~BinaryTree() // Destructor
        { destroySubTree(root); }
    void insertNode(T);
    bool searchNode(T);
    void remove(T);
    void displayInOrder()
        { displayInOrder(root); }
    void displayPreOrder()
        { displayPreOrder(root); }
    void displayPostOrder()
        { displayPostOrder(root); }

    int numNodes();
};
```

```

//*****
// insert accepts a TreeNode pointer and a pointer to a node. The function inserts the *
// node into the tree pointed to by the TreeNode pointer. This function is called recursively. *
//*****

```

```

template <class T>
void BinaryTree<T>::insert(TreeNode *&nodePtr, TreeNode *&newNode)
{
    if (nodePtr == nullptr)
    {
        // Insert the node.
        nodePtr = newNode;
    }
    else if (newNode->value < nodePtr->value)
    {
        // Search the left branch
        insert(nodePtr->left, newNode);
    }
    else
    {
        // Search the right branch
        insert(nodePtr->right, newNode);
    }
}

```

```

//*****
// insertNode creates a new node to hold num as its value, *
// and passes it to the insert function. *
//*****

```

```

template <class T>
void BinaryTree<T>::insertNode(T num)
{
    TreeNode *newNode = nullptr; // Pointer to a new node.

    // Create a new node and store num in it.
    newNode = new TreeNode;
    newNode->value = num;
    newNode->left = newNode->right = nullptr;

    // Insert the node.
    insert(root, newNode);
}

```

```

//*****

```

```
// destroySubTree is called by the destructor. It  *
// deletes all nodes in the tree.                *
//*****

template <class T>
void BinaryTree<T>::destroySubTree(TreeNode *nodePtr)
{
    if (nodePtr->left)
    {
        destroySubTree(nodePtr->left);
    }

    if (nodePtr->right)
    {
        destroySubTree(nodePtr->right);
    }

    delete nodePtr;
}

//*****
// searchNode determines if a value is present in the tree. If so, *
// the function returns true. Otherwise, it returns false.  *
//*****

template <class T>
bool BinaryTree<T>::searchNode(T num)
{
    bool status = false;

    TreeNode *nodePtr = root;

    while (nodePtr)
    {
        if (nodePtr->value == num)
        {
            status = true;
        }
        else if (num < nodePtr->value)
        {
            nodePtr = nodePtr->left;
        }
        else
        {
            nodePtr = nodePtr->right;
        }
    }
}
```

```

        }
    }

    return status;
}

//*****
// remove calls deleteNode to delete the      *
// node whose value member is the same as num. *
//*****

template <class T>
void BinaryTree<T>::remove(T num)
{
    deleteNode(num, root);
}

//*****
// deleteNode deletes the node whose value  *
// member is the same as num.                *
//*****

template <class T>
void BinaryTree<T>::deleteNode(T num, TreeNode *&nodePtr)
{
    if (num < nodePtr->value)
    {
        deleteNode(num, nodePtr->left);
    }
    else if (num > nodePtr->value)
    {
        deleteNode(num, nodePtr->right);
    }
    else
    {
        makeDeletion(nodePtr);
    }
}

//*****
// makeDeletion takes a reference to a pointer to the node that is to be deleted. *
// The node is removed and the branches of the tree below the node are reattached.  *
//*****

template <class T>

```

```

void BinaryTree<T>::makeDeletion(TreeNode *&nodePtr)
{
    // Temporary pointer, used in reattaching the left subtree.
    TreeNode *tempNodePtr = nullptr;

    if (nodePtr == nullptr)
    {
        cout << "Cannot delete empty node.\n";
    }

    else if (nodePtr->right == nullptr)
    {
        tempNodePtr = nodePtr;
        nodePtr = nodePtr->left; // Reattach the left child
        delete tempNodePtr;
    }
    else if (nodePtr->left == nullptr)
    {
        tempNodePtr = nodePtr;
        nodePtr = nodePtr->right; // Reattach the right child
        delete tempNodePtr;
    }
    // If the node has two children.
    else
    {
        // Move one node the right.
        tempNodePtr = nodePtr->right;

        // Go to the end left node.
        while (tempNodePtr->left)
        {
            tempNodePtr = tempNodePtr->left;
        }

        // Reattach the left subtree.
        tempNodePtr->left = nodePtr->left;
        tempNodePtr = nodePtr;

        // Reattach the right subtree.
        nodePtr = nodePtr->right;
        delete tempNodePtr;
    }
}

//*****

```

```
// The displayInOrder member function displays the values      *
// in the subtree pointed to by nodePtr, via inorder traversal. *
//*****
```

```
template <class T>
void BinaryTree<T>::displayInOrder(TreeNode *nodePtr)
{
    if (nodePtr)
    {
        displayInOrder(nodePtr->left);
        cout << nodePtr->value << " "; // endl;
        displayInOrder(nodePtr->right);
    }
}
```

```
//*****
// The displayPreOrder member function displays the values      *
// in the subtree pointed to by nodePtr, via preorder traversal. *
//*****
```

```
template <class T>
void BinaryTree<T>::displayPreOrder(TreeNode *nodePtr)
{
    if (nodePtr)
    {
        cout << nodePtr->value << endl;
        displayPreOrder(nodePtr->left);
        displayPreOrder(nodePtr->right);
    }
}
```

```
//*****
// The displayPostOrder member function displays the values      *
// in the subtree pointed to by nodePtr, via postorder traversal.*
//*****
```

```
template <class T>
void BinaryTree<T>::displayPostOrder(TreeNode *nodePtr)
{
    if (nodePtr)
    {
        displayPostOrder(nodePtr->left);
        displayPostOrder(nodePtr->right);
        cout << nodePtr->value << " "; // endl;
    }
}
```

```

}

//*****
// The numNodes function returns the number of nodes in the tree.*
//*****

template <class T>
int BinaryTree<T>::numNodes()
{
    return countNodes(root);
}

//*****
// The countNodes function uses recursion to count the nodes in the tree.
// This function is called by the public member function numNodes.
//*****

template <class T>
int BinaryTree<T>::countNodes(TreeNode *&nodePtr)
{
    int count;

    if (nodePtr == nullptr)
    {
        count = 0;
    }
    else
    {
        count = 1 + countNodes(nodePtr->left) + countNodes(nodePtr->right);
    }

    return count;
}
#endif

```